

TCP Server

TCP Server

- 1. TCP Server Overview
- 2. Core Concepts
 - 2.1 TCP Server Processing Flow
 - 2.2 Single Client vs Multi-Client
- 3. Hardware Dependencies
- 4. Project Architecture and Flow
- 5. Source Code Details
 - 5.1 Configuration and Global Variables
 - 5.2 `setup()` — Start Server
 - 5.3 `loop()` — State Machine Processing
- 6. Operation Flow
 - 6.1 Build and Flash
 - 6.2 Operation Steps
 - 6.3 Verification Methods
- 7. Common Issues

1. TCP Server Overview

TCP (Transmission Control Protocol) is a **connection-oriented, reliable, byte-stream** transport layer protocol. It establishes connections through three-way handshake, ensuring data arrives in order and intact.

TCP is the foundation of IoT communication. Whether it's HTTP web requests, MQTT message publishing, OTA firmware upgrades, or database remote connections, all rely on TCP's reliable transmission underneath. In industrial automation, smart home, and remote monitoring scenarios, devices report sensor data and receive control commands, requiring "no data loss, no order disruption" transmission guarantees—this is TCP's core value. UDP is faster but unreliable; TCP has the overhead of three-way handshake, but is worth it for the **end-to-end acknowledgment and retransmission mechanism**, which is irreplaceable in most IoT scenarios.

This experiment uses the `WiFiServer` class to wrap lwIP's TCP listen socket, adopting a single-client serial model—each processing loop only serves one client, waiting for new connections after disconnection.

Typical Application Scenarios:

Application Type	Example
Web Server	HTTP Server listening on port 80
IoT Gateway	Receiving TCP connections from multiple sensor nodes
Remote Control	Telnet command line, custom protocols

2. Core Concepts

2.1 TCP Server Processing Flow

From startup to shutdown, a TCP server goes through 6 stages, each corresponding to an Arduino API:

socket()	create socket
↓	wiFiServer server(8080) done automatically during construction
bind(port)	bind port
↓	server.begin() internally calls bind(8080)
listen()	start listening
↓	server.begin() internally calls listen(), waiting for client connection
accept()	accept connection
↓	server.available() is equivalent to accept(), non-blocking
↓	returns WiFiClient object when new connection, returns empty object when no new connection
read / write	send and receive data
↓	read and write through WiFiClient object returned by accept
close()	close connection
	client.stop() release socket, complete four-way handshake

Method	Class	Function	Blocking?
server.begin()	WiFiServer	Bind port and start listening (bind + listen)	No
server.available()	WiFiServer	Accept new client connection (accept)	No
client.available()	WiFiClient	Query readable bytes in receive buffer	No
client.read(buf, len)	WiFiClient	Read data from receive buffer	No
client.write(buf, len)	WiFiClient	Send data to this client	No
client.connected()	WiFiClient	Check if client is still online	No
client.stop()	WiFiClient	Close connection with this client	No

2.2 Single Client vs Multi-Client

This experiment only maintains one `WiFiClient`, which is a **single-client serial model**—only serves one client at a time, and can only accept new connections after the current client disconnects.

```
// Single Client: One Variable
WiFiClient client;
client = server.available();

// Multiple Clients: Array Polling
WiFiClient clients[5];
for (int i = 0; i < 5; i++) {
  if (clients[i] && clients[i].connected()) {
    // Independent reading and writing for each client
  }
}
```

Single client model is suitable for teaching demonstrations; actual projects should use arrays to support multiple concurrent clients.

3. Hardware Dependencies

Pure software experiment. Must first connect to router via WIFI-STA.

4. Project Architecture and Flow

```
setup()
├─ connectWiFi()
└─ server.begin()           // bind + listen

loop()
├─ hasClient=0 → server.available() try accept → 0→1 print Connected
├─ hasClient=1 → while(client.available()) drain data → reply
├─ write() failed → 1→0 print disconnected → stop()
└─ new connection → old client 1→0 print disconnected → new client 0→1
```

5. Source Code Details

Complete source code: [Source code](#) → [Arduino](#) → [3.Network Communication](#) → [TCP_Server](#) → [TCP_Server.ino](#)

5.1 Configuration and Global Variables

SSID and password can be modified according to actual environment. `hasClient` tracks client count (0=none, 1=connected), used for state machine control and disconnect detection.

```
#include <WiFi.h>
#include <lwip/sockets.h>    // select() / FIONREAD / fd() / select, FIONREAD, raw fd access

// ===== Config =====
const char* WIFI_SSID = "Yahboom2";      // WiFi SSID
const char* WIFI_PASS = "yahboom890729"; // WiFi password
```

```
#define TCP_PORT          8080                // TCP listen port

WiFiServer server(TCP_PORT);                 // TCP server object
WiFiClient client;                           // current client object
int hasClient = 0;                           // client count 0=none 1=connected

char recv_buf[128];                          // receive buffer
char send_buf[128];                          // send buffer
```

5.2 `setup()` — Start Server

After serial initialization, switch to STA mode → connect WiFi → `server.begin()` completes bind + listen.

```
void setup() {
  Serial.begin(115200);
  delay(1000);

  WiFi.mode(WIFI_STA);           // station mode
  connectWiFi();                  // connect to WiFi
  server.begin();                  // bind + listen, start server

  Serial.println("=====");
  Serial.println(" TCP Server Startup successful!");
  Serial.print(" Server IP: ");
  Serial.println(WiFi.localIP()); // print server IP
  Serial.print(" Port: ");
  Serial.println(TCP_PORT);        // print listen port
  Serial.println("=====");
}
```

5.3 `loop()` — State Machine Processing

The main loop has three trigger paths to detect client disconnection: ① `server.available()` new connection replaces old client ② `client.write()` reply failure ③ `select()+FIONREAD` detects FIN half-close state.

`hasClient` ensures $0 \rightarrow 1$ prints Connected, $1 \rightarrow 0$ prints disconnected.

```
void loop() {  
  // ===== WiFi watchdog: auto reconnect =====  
  if (WiFi.status() != WL_CONNECTED) {  
    connectWiFi();  
  }  
  
  // ===== Trigger 1: new connection → replace old client =====  
  WiFiClient newClient = server.available(); // non-blocking accept  
  if (newClient) {  
    if (hasClient == 1) { // old client still present  
      Serial.println("Client disconnected"); // 1→0: print disconnect  
      client.stop(); // release old socket  
    }  
    client = newClient; // hold new connection  
    hasClient = 1; // 0→1: mark client present  
  }  
}
```

```

Serial.println("Client Connected");
}

if (hasClient == 0) {                                // no client, wait next loop
    delay(100);
    return;
}

// ===== Trigger 2: exchange → write fail → disconnect =====
while (client.available()) {                          // drain internal buffer
    char buf[128];
    int len = client.read((uint8_t*)buf, sizeof(buf) - 1); // read data
    if (len > 0) {
        buf[len] = '\0';                                // null terminate
        Serial.printf("Received: %s\n", buf);

        char reply[160];
        sprintf(reply, "Server OK: %s", buf); // build reply
        if (client.write((const uint8_t*)reply, strlen(reply)) <= 0) {
            // send failed → disconnect
            Serial.println("Client disconnected"); // print disconnect
            client.stop();                        // release socket
            hasClient = 0;                        // count back to 0
            client = WiFiClient();               // reset to empty
            return;
        }
        Serial.printf("Reply: %s\n", reply);
    }
}

// ===== Trigger 3: FIN detection =====
int fd = client.fd();                                // get raw socket fd
if (fd < 0) {                                          // invalid fd = disconnected
    Serial.println("Client disconnected");
    client.stop();
    hasClient = 0;
    client = WiFiClient();
    return;
}

fd_set readSet;
FD_ZERO(&readSet);
FD_SET(fd, &readSet);                                // watch fd for readability
struct timeval tv = {0, 0};                          // zero wait = poll only
if (select(fd + 1, &readSet, NULL, NULL, &tv) > 0) { // socket readable
    int pending = 0;
    lwip_ioctl(fd, FIONREAD, &pending);               // query pending bytes
    if (pending == 0) {                                // readable + 0 bytes = FIN
        Serial.println("Client disconnected"); // instant disconnect
        client.stop();
        hasClient = 0;
        client = WiFiClient();
        return;
    }
}

```

```

    }
}

delay(10); // 10ms polling interval
}

```

Three disconnect trigger paths: ① New connection replacement ② write() send failure ③ select+FIONREAD detects FIN. `hasClient` count ensures clear state, 0→1 prints Connected, 1→0 prints disconnected.

6. Operation Flow

6.1 Build and Flash

Arduino flash process: See `Arduino → 1. Before Development → 2. Build and Flash`.

1. Build and flash
2. Use network debugging assistant or TCP client to connect to `Server IP:8080`

6.2 Operation Steps

Step 1: Flash program, record server IP

Build and flash to ESP32-S3 development board. Open serial monitor (baud rate 115200), wait for WiFi connection success, and note down the server IP address printed in serial:

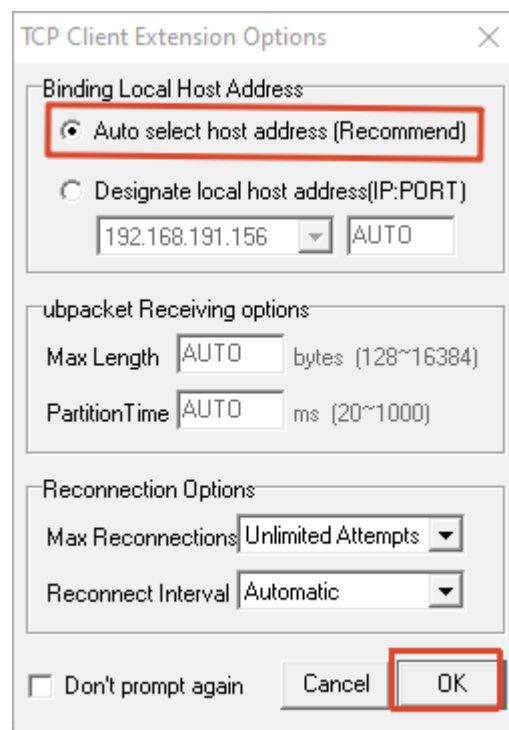
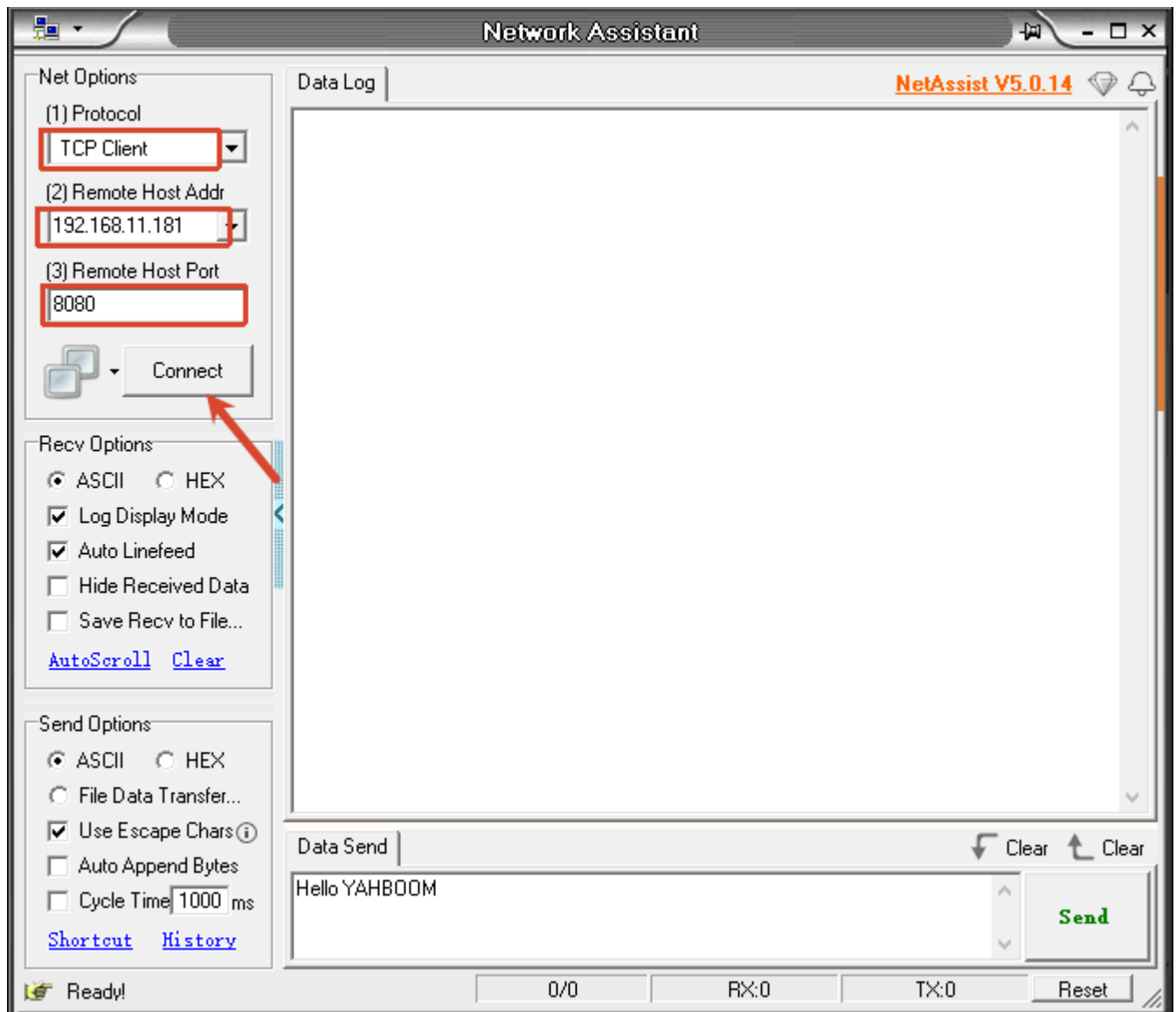
```

Connecting to WiFi: Yahboom2 (Attempt 1)
...
WiFi Connected successfully!
Server IP: 192.168.11.181
=====
TCP Server Startup successful!
Server IP: 192.168.11.181
Port: 8080
=====

```

Step 2: Open network debugging assistant as TCP client

Open network debugging assistant on PC , configure with the following parameters:



Parameter	Value
Protocol Type	TCP Client
Remote Host	ESP32 server IP recorded in Step 1 (e.g., 192.168.11.229)
Remote Port	8080

Step 3: Send data, observe echo

Enter `Hello YAHBOOM` in the send area of the network debugging assistant and click send. After ESP32 receives it, it automatically replies `Server OK: Hello YAHBOOM`, and the serial monitor prints:

```
Connecting to WiFi: Yahboom2 (Attempt 1)
...
WiFi Connected successfully!
Server IP: 192.168.11.181
=====
TCP Server Startup successful!
Server IP: 192.168.11.181
Port: 8080
=====
Client Connected
Received: Hello YAHBOOM
Reply: Server OK: Hello YAHBOOM
Received: Hello YAHBOOM
Reply: Server OK: Hello YAHBOOM
```

Network debugging assistant prints:

```
[2026-08-06 19:39:37.628]# Connecting to server...
[2026-08-06 19:39:37.691]# Server connected from local 192.168.11.59:53413
[2026-08-06 19:39:38.826]# SEND ASCII/13 >>>
Hello YAHBOOM
[2026-08-06 19:39:38.938]# RECV ASCII/24 from SERVER <<<
Server OK: Hello YAHBOOM
[2026-08-06 19:39:39.009]# SEND ASCII/13 >>>
Hello YAHBOOM
[2026-08-06 19:39:39.039]# RECV ASCII/24 from SERVER <<<
Server OK: Hello YAHBOOM
[2026-08-06 19:39:40.117]# SEND ASCII/13 >>>
Hello YAHBOOM
[2026-08-06 19:39:40.258]# RECV ASCII/24 from SERVER <<<
Server OK: Hello YAHBOOM
```


Received: Hello YAHBOOM `, // serial monitor prints:
Reply: Server OK: Hello YAHBOOM `, // serial monitor prints:

Step 4: Test disconnect and reconnect

Click "Disconnect" in the network debugging assistant. Serial monitor prints `Client disconnected`. Click "Connect" again, ESP32 automatically accepts new client, prints `Client Connected`, and restores normal send/receive.

`Client disconnected`

6.3 Verification Methods

Operation	Expected Result
Client connects and sends data	Receives "Server OK: "
Client disconnects	Prints "Client disconnected"
New client connects	Auto accepts, prints "Client Connected"

7. Common Issues

Phenomenon	Cause	Solution
Client disconnects but doesn't print disconnected	WiFiClient::connected() is unreliable for normal FIN	Use <code>hasClient</code> count + <code>write()</code> return value detection + <code>server.available()</code> new connection replacement
Client connects but gets no reply	Server busy or occupied by other client	Check if single client model is blocking
<code>server.available()</code> returns empty	No new connection	Normal—non-blocking design, just continue polling
Multiple clients but can only serve one	Single client object limitation	Change to <code>WiFiClient clients[5]</code> array model